Discussion 4: Image Filtering

Discussion Topic:

Of the three types of image filtering discussed in this module, which is your favorite and why? Additionally, describe your preferred method of image filtering if the type you chose has multiple methods of application.

My Post:

Hello Class,

If I have to choose an image filtering technique discussed in this module, I would have to choose high-pass filtering, more especially Laplacian filtering method. Low-pass filtering is useful when the goal is to smooth an image or remove noise, but I find high-pass filtering more interesting (to me) because it does almost the opposite. Instead of suppressing rapid changes in pixel intensity, it keeps or emphasizes them. Those rapid changes usually occur around edges and fine details in an image. The module (Colorado State University Global, n.d.) explains that high-pass filtering increases the contrast between light and dark regions and brings out image features.

The Laplacian is a second-derivative operation that responds strongly (highlights) where pixel intensity changes quickly. In other words, relatively uniform areas produce little response, while edges stand out. A second-derivative operation in the context of image filtering and when using the Laplacian method implies that it is a first derivative that measures how pixel intensity changes from one pixel to the next, and a second derivative that measures how that change itself changes…
What I find interesting about the Laplacian filtering method is that it uses a small kernel to extract structural information that is difficult to see just by looking at the original pixel values. These Laplacian kernels can also be positive and negative, which reverse the signs of the center and neighboring values depending on how the operator is applied negatively or positively.

For example, the following is not the Laplacian method, but a simple high-pass sharpening method:

```
import cv2
import numpy as np

# Load the input image from the current working directory.
image = cv2.imread("image.jpg")

# Define a 3 x 3 sharpening kernel.
# The center pixel is multiplied by 5.
# The pixels directly above, below, left, and right are multiplied by -1.
# This increases the contrast between neighboring pixels and makes edges
# and fine details appear sharper.
kernel = np.array([
    [ 0, -1,  0],
    [-1,  5, -1],
    [ 0, -1,  0]
])

# Apply the sharpening kernel to the image using 2D convolution.
```

```
# -1 tells OpenCV to keep the output image at the same depth
# as the original image.
sharpened = cv2.filter2D(image, -1, kernel)
```

(OpenCV with Python, n.d.)

To understand what the high-pass sharpening method is doing, we need to look at the kernel itself. The center pixel is multiplied by 5, while the pixels directly above, below, left, and right are subtracted. If the surrounding pixels are similar to the center pixel, much of that subtraction cancels out. If there is a large intensity difference, such as at an edge, the difference becomes stronger in the output.

OpenCV also provides the Laplacian method `with cv2.Laplacian()`, which I would use to detect or examine edges:

```
# Apply a Gaussian blur to reduce image noise before edge detection.
blurred = cv2.GaussianBlur(image, (7, 7), 10)

# Convert the blurred BGR image to grayscale.
gray = cv2.cvtColor(blurred, cv2.COLOR_BGR2GRAY)

# Apply the Laplacian method to detect intensity changes and edges.
laplacian = cv2.Laplacian(gray, cv2.CV_32F)
```

Note that a blur needs to be applied before using `cv2.Laplacian()` function. Here, the Gaussian blur was applied first because the Laplacian does not know the difference between an important edge and a small intensity change caused by noise. Noise also contains high-frequency information, and the Laplacian method can make this noise more noticeable. Applying smoothing/blurring to an image before applying the Laplacian method allows reducing noise; otherwise, the resulting image may contain too much 'unwanted' detail or amplified noise.

Additionally, I found that neural-network approaches such as a denoising autoencoder are very interesting. For example, a basic denoising autoencoder learns to reconstruct a clean image from a noisy input. Or RED-Net, which is a fully convolutional Residual Encoder-Decoder Neural Network that uses convolutional layers to eliminate corruptions and transposed-convolutional layers to recover image details, to learn an end-to-end mapping from corrupted images to restored images (Dey, 2025).

-Alex

References:

Colorado State University Global. (n.d.). Module 4: Image filtering [Interactive lecture]. Canvas.

Dey, S. (2025). *Image processing and computer vision masterclass with Python: Learn advanced image processing techniques, computer vision fundamentals, and applications* (2nd ed.). BPB Publications.

OpenCV. (n.d.). *Smoothing images*. OpenCV documentation. https://docs.opencv.org/4.13.0/d4/d13/tutorial_py_filtering.html